- **为什么学习JavaScript**
  - JavaScript的Web标准构成之一，是前端的基石，作为前端工程师，必须熟练掌握
  - 前端数据可视化的核心（webgl，echarts...）
  - Nodejs的核心
  - 小程序、Vuejs、Reactjs等框架的核心
  - 网页游戏
  - ...

- **JavaScript 是什么**
  - Web 标准的构成

标准	内容	说明
结构	HTML	网页元素的结构和内容
表现	CSS	网页元素的外观和位置，包括：版式、颜色、大小等
行为	JavaScript	网页模型的定义与交互

  - JavaScript 是一种编程语言，简称JS，可以用来创建动态更新的内容，控制多媒体，制作图像动画等等实现人机交互效果，简单说，可以通过简短的代码来实现神奇的功能。

- **JavaScript 书写位置**
  - 内部 JavaScript
    - 直接写在html文件里，用script标签包住
    - 规范：script标签写在</body>上面
    - 拓展： alert('你好，js'）页面弹出警告对话框

```html
<body>
  <!-- 内联形式：通过 script 标签包裹 JavaScript 代码 -->
  <script>
    alert('嗨，欢迎来传智播学习前端技术！');
  </script>
</body>
```

- 外部 JavaScript
  - 代码写在以.js结尾的文件里
  - 语法：通过script标签，引入到html页面中
  - 注意： script标签中间无需写代码，否则会被忽略！

```html
<body>
  <!-- 外联形式：通过 script 的 src 属性引入独立的 .js 文件 -->
  <script src="demo.js"></script>
</body>
```

- **JavaScript 注释**
  - 单行注释
    - 符号：//
    - 作用：//右边这一行的代码会被忽略
    - 快捷键：ctrl + /

```html
<script>
  // 这种是单行注释的语法
  // 一次只能注释一行
  // 可以重复注释
</script>
```

  - 块注释
    - 符号：/* */
    - 作用：在/* 和 */ 之间的所有内容都会被忽略
    - 快捷键：shift + alt + A

```html
<script>
  /* 这种的是多行注释的语法 */
  /*
    更常见的多行注释是这种写法
    在些可以任意换行
    多少行都可以
  */
</script>
```

- **JavaScript 结束符**

- 代表语句结束
- 英文分号；
- 可写可不写（现在不写结束符的程序员越来越多）
- 换行符（回车）会被识别成结束符 ,因此在实际开发中有许多人主张书写 JavaScript 代码时省略结束符
- 但为了风格统一，要写结束符就每句都写，要么每句都不写（按照团队要求.）

```
<script>
  alert(1);
  alert(2);
</script>
```

```
<script>
  alert(1)
  alert(2)
</script>
```

- **JavaScript 输入输出语句**
  - 输出语句：
    - 第一种：

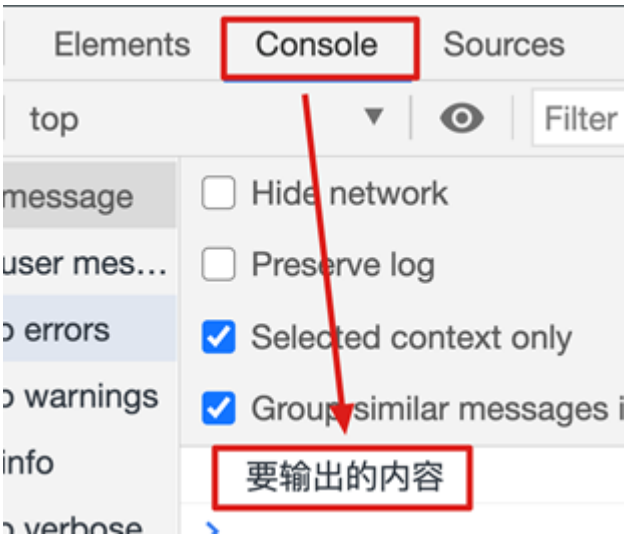
```
document.write('要输出的内容');
```

      - 向body内输出内容
      - 如果输出的内容写的是标签，也会被解析成网页元素
    - 第二种：

```
alert('要输出的内容');
```

      - 页面弹出警告对话框
    - 第三种：

```
console.log('要输出的内容')
```



- 浏览器的控制台输出
- 快捷键
  - win: Ctrl + Shift + i
  - mac: Command + Options + i

- 输入语句：

```
prompt('请输入您的姓名:');
```

  - 显示一个对话框，对话框中包含一条文字信息，用来提示用户输入文字

- **变量的基本使用**
  - 声明变量：
    - 要想使用变量，首先需要创建变量（专业说法： 声明变量）
    - 语法：

```
let 变量名
```

      - 声明变量有两部分构成：声明关键字、变量名（标识）
      - let 即关键字 (let: 允许、许可、让、要)，所谓关键字是系统提供的专门用来声明（定义）变量的词语
    - 举例：

```
let age;
```

    - age 即变量的名称，也叫标识符
  - 变量赋值：
    - 定义了一个变量后，你就能够初始化它（赋值）。在变量名之后跟上一个“=”，然后是数值。

```
let age;
// 赋值，将 18 这个数据存入了 age 这个“容器”中
age = 18;
// 这样 age 的值就成了 18
document.write(age);
```

- 注意：是通过变量名来获得变量里面的数据。
- 也可以声明变量的时候同时给变量初始化。

```
<script>
    // 声明了一个age变量，同时里面存放了 18 这个数据
    let age = 18
</script>
```

- 更新变量：
  - 变量赋值后，还可以通过简单地给它一个不同的值来更新它。

```
// 声明了一个age变量，同时里面存放了 18 这个数据
let age = 18
// 变量里面的数据发生变化更改为 19
age = 19
// 页面输出的结果为 19
document.write(age)
```

- 注意： let 不允许多次声明一个变量。

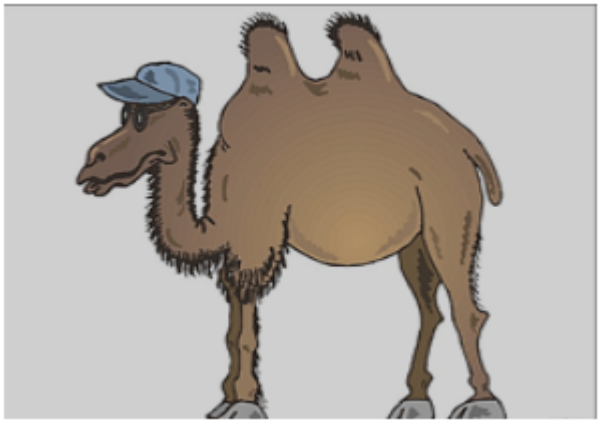
- **变量的注意事项**
  - 不要多次声明同一个变量，否则会报错

```
// 声明了一个age变量，同时里面存
let age = 18
// 这里不允许多次声明一个变量，
let age = 19
// 输出会报错
document.write(age)
```

- 修改变量，直接对变量重新赋值即可
- 之前声明变量用va关键字，现在用let

- **变量的本质**
  - 内存：计算机中存储数据的地方，相当于一个空间
  - 变量：是程序在内存中申请的一块用来存放数据的小空间

- **变量命名规则与规范**
  - 规则：
    - 不能用关键字
      - 关键字：有特殊含义的字符，JavaScript 内置的一些英语词汇。例如：let、var、if、for等
    - 只能用下划线、字母、数字、$组成，且数字不能开头
    - 字母严格区分大小写，如 Age 和 age 是不同的变量
  - 规范：
    - 起名要有意义
    - 遵守驼峰命名法
      - 第一个单词首字母小写，后面每个单词首字母大写。例：userName



- **数据类型**

数据类型	作用	表示形式	例
string（字符串）	表示文字内容	单引号或者双引号包起来	'传智' "黑马"
number（数字）	表示数字内容	直接写数字	10,11,12,10.32
boolean（布尔）	表示对立的两种状态：真和假、对和错、开和关等	true和false	true，false
undefined（未定义）	表示未定义	undefined	undefined

  - 此图不全，还有：bigint（巨数）、null（空引用）、symbol（符号），共7种基本类型。

- **数据类型 – 数字（值）类型（number）**
  - 即我们数学中学习到的数字，可以是整数、小数、正数、负数。

```
let score = 100; // 正整数
let price = 12.345; // 小数
let temperature = -40; // 负数
```

  - JavaScript 中的数值类型与数学中的数字是一样的，分为正数、负数、小数等。

- **数据类型 – 字符串类型（string）**
  - 通过单引号（''）、双引号（""）或反引号（``）包裹的数据都叫字符串，单引号和双引号没有本质上的区别，推荐使用单引号。

```
let user_name = '小明'; // 使用单引号
let gender = "男"; // 使用双引号
let str = '123'; // 看上去是数字，但是用引号包裹了就成了字符串了
let str1 = ''; // 这种情况叫空字符串
```

  - 注意事项：
    - 无论单引号或是双引号必须成对使用
    - 单引号/双引号可以互相嵌套，但是不以自已嵌套自已（口诀：外双内单，或者外单内双）
    - 必要时可以使用转义符 \，输出单引号或双引号

- **数据类型 – 布尔类型（boolean）**
  - 表示肯定或否定时在计算机中对应的是布尔类型数据。
  - 它有两个固定的值 true 和 false，表示肯定的数据用 true（真），表示否定的数据用 false（假）。

```
// pink老师帅不帅? 回答 是 或 否
let isCool = true; // 是的，摔死了!
isCool = false; // 不，套马杆的汉子!
```

- **数据类型 – 未定义类型（undefined）**
  - 未定义是比较特殊的类型，只有一个值 undefined。
  - 只声明变量，不赋值的情况下，变量的默认值为 undefined，一般很少【直接】为某个变量赋值为 undefined。

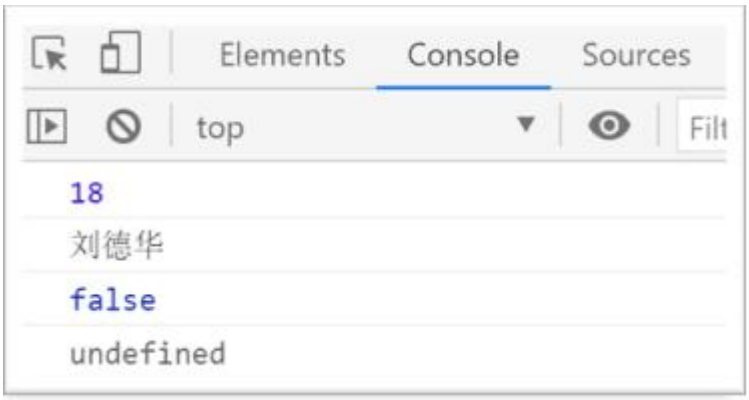
```
let age     // 声明变量但是未赋值
document.write(age)  // 输出 undefined
```

情况	说明	结果
var age ; console.log (age);	只声明 不赋值	undefined
console.log(age)	不声明 不赋值 直接使用	报错
age = 10; console.log (age);	不声明 只赋值	10（不提倡）

- **数据类型总结**
  - 基本数据类型（7种）：number：数字（可用typeof分辨，值为number）、bigint：巨数（可用typeof分辨，值为bigint）、string：字符串（可用typeof分辨，值为string）、boolean：布尔（可用typeof分辨，值为boolean）、undefined：未定义（可用typeof分辨，值为undefined）、null：空引用（不可用typeof分辨，值为object）、symbol：符号（可用typeof分辨，值为symbol）。
  - 引用数据类型：对象（可用typeof分辨，值为object）、数组（object子类，不可用typeof分辨，值为object，可用Array.isArray()分辨）、函数（object子类，可用typeof分辨，值为function）等。
  - 特殊数据：
    - undefined：
      - 类型：未定义（undefined）
      - 说明：一个声明未定义的变量的初始值，或没有实际参数的形式参数，是类型undefined的唯一值。
    - null：
      - 类型：空引用（null）
      - 说明：表示一个不存在或者无效object或者地址引用，是类型null的唯一值。
    - NaN：
      - 类型：数字（number）
      - 说明：非数值，Not-a-Number，是类型number的特殊值之一，其他特殊值还有+Infinity、-Infinity。
  - 基本类型包装对象
    - 除了null和undefined之外，所有基本类型都有其对应的包装对象
      - String为字符串基本类型
      - Number为数值基本类型
      - BigInt为大整数基本类型
      - Boolean为布尔基本类型
      - Symbol为字面量基本类型
    - 这个包裹对象的valueOf()方法返回基本类型值

- **检测数据类型**
  - 控制台输出语句：

```
let age = 18
let uname = '刘德华'
let flag = false
let buy
console.log(age)
console.log(uname)
console.log(flag)
console.log(buy)
```



    - 控制台语句经常用于测试结果来使用。
    - 可以看出数字型和布尔型颜色为蓝色，字符串和undefined颜色为灰色
  - 通过 typeof 关键字检测数据类型

```
let age = 18
let uname = '刘德华'
let flag = false
let buy
console.log(typeof age)   // number
console.log(typeof uname) // string
console.log(typeof flag)  // boolean
console.log(typeof buy)   // undefind
```

- **算数运算符**
  - 数学运算符也叫算术运算符，主要包括加、减、乘、除、取余（求模）。
    - +：求和
    - -：求差
    - *：求积
    - /：求商
    - %：取模（取余数）

- **优先级**
  - JavaScript中 优先级越高越先被执行，优先级相同时以书写顺序从左向右执行。

        - 乘、除、取余优先级相同
        - 加、减优先级相同
        - 乘、除、取余优先级大于加、减
        - 使用 () 可以提升优先级

- **+ 运算符**
    - + 运算符在数字型(number)中是求和运算

```
let boy = 30
let girl = 31
console.log(boy + girl)   // 输出 61
```

    - + 运算符在字符串型(string)中是拼接

```
document.write('我叫' + '刘德华')    // 我叫刘德华

let uname = '刘德华'
let song = '忘情水'
document.write(uname + song)  // 刘德华忘情水
```

- **模板字符串**
    - 作用
        - 拼接字符串和变量
        - 在没有它之前，要拼接变量比较麻烦

```
document.write('大家好，我叫' + name + '，今年' + age + '岁')
```

    - 符号
        - ``
        - 在英文输入模式下按键盘的tab键上方那个键（1左边那个键）
        - 内容拼接变量时，用 ${} 包住变量

```
document.write(`大家好，我叫${name}，今年${age}岁`)
```

- **显式转换**
    - 概念：
        - 自己写代码告诉系统该转成什么类型
    - 转换方法：
        - Number(数据)
            - 转成数值类型

        - 如果字符串内容里有非数字，转换失败时结果为 NaN（Not a Number）即不是一个数字
        - NaN也是number类型的数据，代表非数字
    - Boolean（数据）
        - 转成布尔类型
        - 0、NaN、空字符串、undefined、null（3空2数字）转成false,其他都是true
    - String（数据）
        - 把数字转为字符串

- **隐式转换**
    - 概念：
        - 某些运算符被执行时，系统内部自动将数据类型进行转换，这种转换称为隐式转换。
    - 规则：
        - +号两边只要有一个是字符串，都会把另外一个转成字符串
        - 除了+以外的所有算术运算符都会把数据转成数值类型
    - 缺点：
        - 转换类型不明确，靠经验才能总结
    - 两个例子：
        - 数字加上空字符串转换成字符串

```
let str = 666 + ''
```

        - 字符串前面（后面不行）加上加号转换成数字

```
let num = +'666'
```